

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



ADVANCED DATA STRUCTURES AAT REPORT on

Multi-Layered Image Steganalysis Using AVL Trees *Submitted by*

Owais Ali Khadim Batti (1BM23CS221)
Megha C H (1BM22CS188)
Namita D A (1BM22CS202)
Parth Patil (1BM23CS227)

Under the Guidance of
Prof. Rajeshwari K
Assistant Professor, BMSCE

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February-May 2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visveswaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the AAT work entitled “**Multi-Layered Image Steganalysis Using AVL Trees**” is carried out by **Owais Ali Khadim Batti (1BM23CS221), Megha C H (1BM22CS188), Namita D A (1BM22CS202), Parth Patil (1BM23CS227)** who are bonafide students of **B.M.S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visveswaraya Technological University, Belgaum during the year 2025-2026. The AAT report has been approved as it satisfies the academic requirements in respect of **Advanced Data Structures (23CS6PEADS)** work prescribed for the said degree.

Signature of the Guide
Prof. Rajeshwari K.
Assistant Professor
BMSCE, Bengaluru

Signature of the HOD
Dr. Kavitha Sooda
Prof. & Head, Dept. of CSE
BMSCE, Bengaluru

B.M.S. COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

We, Owais Ali Khadim Batti (1BM23CS221), Megha C H (1BM22CS188), Namita D A (1BM22CS202) and Parth Patil (1BM23CS227) students of 6th Semester, B.E, Department of Computer Science and Engineering, B. M. S. College of Engineering, Bangalore, hereby declare that, this AAT entitled "Multi-Layered Image Steganalysis Using AVL Trees" has been carried out by us under the guidance of Prof. Anusha S Assistant Professor, Department of CSE, B. M. S. College of Engineering, Bangalore during the academic semester February to May 2025.

We also declare that to the best of our knowledge and belief, the development reported here is not from part of any other report by any other students.

Signature

Owais Ali Khadim Batti (1BM23CS221),

Megha C H (1BM22CS188),

Namita D A (1BM22CS202)

Parth Patil (1BM23CS227)

TABLE OF CONTENTS

Chapter no.	Title	Page no.
1	PROBLEM STATEMENT	6
1.1	Problem Description	6
1.2	Algorithm Chosen	6
1.3	Motivation	7
1.4	Applications	7
1.5	Work Highlights	7
2	DESIGN	9
2.1	Introduction	9
2.2	Working	9
2.3	Implementation	9
2.4	Relevance	10
2.5	Summary	10
3	ALGORITHM	11
4	RESEARCH PAPER AND DATA STRUCTURES USED	15
5	IMPLEMENTATION	18
5.1	Code	18
5.2	Output	22
6	CONCLUSION	23
7	REFERENCES	24

1. PROBLEM STATEMENT

1.1 Problem Description

Steganography is the practice of concealing a file, message, image, or video within another carrier file. Unlike cryptography, which hides the meaning of a message, steganography hides the very existence of communication. The most widespread embedding technique in the spatial domain is Least Significant Bit (LSB) replacement, where the lowest bit of each pixel intensity value is modified to encode hidden data. Such hidden payloads can bypass standard security filters and are increasingly abused for covert command-and-control, data exfiltration, and malware delivery.

Conventional steganalysis approaches are limited: pure CNN classifiers (e.g., SRNet) miss low payload capacities or non-standard embedding orders, and single-method LSB extractors miss data hidden in specific channels or planes. Reliable detection therefore requires combining deep learning, statistical anomaly tests, and multi-strategy bit extraction, while organizing the recovered payload in an efficient self-balancing data structure for further forensic analysis.

This project develops a multi-layered Steganalysis Detector that fuses an SRM-preprocessed CNN, four LSB extraction strategies, a Chi-Square sample-pair statistical test, and an AVL Tree payload index, exposed through a Streamlit web application.

1.2 Algorithm Chosen

Primary Data Structure: AVL Tree (self-balancing Binary Search Tree)

Supporting Techniques:

- SRM (Spatial Rich Model) high-pass filtering for noise residual extraction
- CNN (SRNet-inspired) binary classifier for stego vs. clean prediction
- Multi-method LSB bit extraction (RGB raster, Red-only, sequential plane, stegano library)
- Chi-Square Sample-Pair Analysis for statistical LSB anomaly detection
- AVL Tree insertion with LL/LR/RL/RR rotations for $O(\log n)$ payload indexing

1.3 Motivation for Choosing This Problem

- High Practical Impact: Steganalysis is critical for cyber-defence, malware analysis, digital forensics, and intellectual-property protection.
- Real-Time Requirement: Detectors must process suspect images within seconds for SOC and incident-response workflows.
- Relevance to Advanced Data Structures: AVL Trees provide guaranteed $O(\log n)$ insertion and ordered traversal of recovered payload characters even when the input is already sorted.
- Innovation Potential: The decision-fusion engine combines learned and statistical signals, dramatically reducing false negatives compared to a single classifier.

By implementing this problem, we gain practical experience in:

- Designing self-balancing tree data structures and rotation logic
- Building deep-learning pipelines with custom non-trainable preprocessing layers
- Combining heuristic, statistical, and learned detectors via decision fusion
- Deploying ML inference behind an interactive web UI

1.4 Applications of Steganalysis

- Cyber-Security: Detect covert exfiltration of credentials or PII through image attachments.
- Digital Forensics: Recover hidden evidence in seized media during criminal investigations.
- Malware Analysis: Identify stego-droppers used by APT groups to smuggle payloads past gateways.
- Content Moderation: Flag hidden illicit material in user-uploaded images on social platforms.
- Intellectual Property Protection: Verify presence/absence of watermarks and hidden ownership tags.
- Email and Messaging Security: Inspect inline images for hidden command channels.

1.5 Work Highlights

- Designed and trained an SRNet-inspired CNN with fixed SRM high-pass preprocessing
- Implemented four independent LSB extraction strategies for maximum recall
- Implemented a normalised Chi-Square Sample-Pair statistical detector ($\chi^2 < 1.5$ threshold)
- Implemented an AVL Tree from scratch (insert, height, balance, four rotations, inorder)
- Built a decision-fusion engine that elevates confidence based on extraction and statistical evidence
- Deployed the full pipeline as an interactive Streamlit web application with forensic reporting

2. DESIGN

2.1 Introduction to the Design

- The chosen design is a Multi-Layered Steganalysis Detector that classifies an uploaded image as Clean or Stego and, when possible, recovers the embedded ASCII payload.
- The system combines a PyTorch CNN (SRNet-inspired) with multi-method LSB extraction, a Chi-Square statistical detector, and an AVL Tree for payload organisation.
- Purpose: Provide high-recall steganography detection with minimal false negatives and a deterministic, sorted view of the recovered payload for downstream forensic processing.

2.2 How the Steganalysis Design Works

- Image Upload: User uploads a PNG/JPG/BMP/TIFF image through the Streamlit UI.
- Preprocessing: Image is resized to 256×256 and normalised into a tensor.
- SRM Filtering: Three fixed high-pass kernels suppress smooth image content and amplify steganographic noise residuals.
- CNN Inference: The SRNet-style network produces a sigmoid probability of being a stego image.
- Multi-Method Extraction: Four LSB extractors run in parallel — stegano-library decode, raw RGB raster, Red-channel only, and sequential plane.
- Statistical Test: A normalised Chi-Square Sample-Pair test inspects (2k, 2k+1) histogram pairs.
- Decision Fusion: If any extractor returns readable text → confidence = 1.0; if $\chi^2 < 1.5$ → confidence ≥ 0.90 ; else fall back to CNN score.
- AVL Indexing: Each character of the recovered payload is inserted into an AVL Tree.
- Reporting: Verdict, confidence, hidden message, AVL inorder traversal, and χ^2 score are displayed.

2.3 How the Steganalysis System Is Made (Design and Implementation)

A. Data Collection and Storage

- Collect a corpus of ≈ 500 clean JPG/PNG images covering varied scenes and textures.
- Generate a balanced dataset by embedding random LSB payloads in half of the images.
- Split the dataset 70/15/15 into training, validation, and test sets.

B. Core Data Structure: AVL Tree

- Each AVL node stores one recovered character plus left/right children and a height field.
- Insertion is $O(\log n)$ and preserves the BST invariant after every operation.
- Imbalance ($|\text{balance}| > 1$) triggers one of four rotations: LL, LR, RL, RR.
- Inorder traversal yields characters in sorted order — useful for frequency analysis and deterministic payload fingerprinting.

C. Detection and Decision Algorithm

- On each upload, the CNN produces a probability $p \in [0, 1]$.
- Four LSB extractors run; first non-empty printable result wins.
- Chi-Square test computes normalised χ^2 over even/odd pixel pairs.
- Decision fusion overrides p when extraction succeeds or when χ^2 falls below the suspicion threshold.

D. User Interface (UI)

- Streamlit single-page app with file uploader, Analyze button, and live status indicators.
- Results panel renders verdict (red/green), confidence metric, χ^2 score, hidden message, and AVL inorder string.
- Sidebar exposes model load status and pipeline diagram.

2.4 Relevance to the Chosen Problem Statement

- **Speeds Up Forensic Triage:** One upload yields a complete verdict in seconds.
- **Improves Detection Accuracy:** Combining CNN, extraction, and statistics reduces false negatives compared to any single method.
- **Enhances User Experience:** Interactive UI exposes confidence, raw χ^2 , and recovered payload for human review.
- **Efficient Payload Indexing:** AVL Tree guarantees $O(\log n)$ insertion even on adversarial (already-sorted) payloads.
- **Supports Auditability:** Each layer's output is reported separately so an analyst can justify the verdict.

2.5 Summary

The Steganalysis Detector designed using a self-balancing AVL Tree and a layered CNN/statistical pipeline effectively addresses the challenge of identifying hidden data inside images. Its multi-layered architecture, deterministic payload organisation, and forensic reporting make it a highly relevant and practical solution for the chosen problem.

3. ALGORITHM FOR THE CHOSEN PROBLEM

AVL Tree implementation used to index the recovered steganographic payload:

```
class AVLNode:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1

class AVLTree:
    def __init__(self):
        self.root = None

    def insert(self, key):
        self.root = self._insert(self.root, key)

    def _insert(self, root, key):
        if not root:
            return AVLNode(key)
        elif key <= root.key:
            root.left = self._insert(root.left, key)
        else:
            root.right = self._insert(root.right, key)

        root.height = 1 + max(self.get_height(root.left),
                               self.get_height(root.right))
        balance = self.get_balance(root)

        # Left Left
        if balance > 1 and key <= root.left.key:
            return self.right_rotate(root)
        # Right Right
        if balance < -1 and key > root.right.key:
            return self.left_rotate(root)
        # Left Right
        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)
        # Right Left
        if balance < -1 and key <= root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)
        return root

    def left_rotate(self, z):
        y = z.right
        T2 = y.left
        y.left = z
        z.right = T2
        z.height = 1 + max(self.get_height(z.left),
                           self.get_height(z.right))
        y.height = 1 + max(self.get_height(y.left),
                           self.get_height(y.right))
```

```

        return y

def right_rotate(self, z):
    y = z.left
    T3 = y.right
    y.right = z
    z.left = T3
    z.height = 1 + max(self.get_height(z.left),
                       self.get_height(z.right))
    y.height = 1 + max(self.get_height(y.left),
                       self.get_height(y.right))
    return y

def get_height(self, root):
    if not root:
        return 0
    return root.height

def get_balance(self, root):
    if not root:
        return 0
    return self.get_height(root.left) - self.get_height(root.right)

def get_inorder(self):
    res = []
    self._inorder(self.root, res)
    return "".join(res)

def _inorder(self, root, res):
    if root:
        self._inorder(root.left, res)
        res.append(root.key)
        self._inorder(root.right, res)

```

Starting from an empty tree, the recovered characters are inserted one by one. After each insertion the tree updates the height field and recomputes balance; if $|\text{balance}| > 1$ the appropriate rotation (LL/LR/RR/RL) is applied to restore the AVL invariant. Inorder traversal of the final tree yields the payload characters in sorted order, providing a deterministic fingerprint of the embedded message regardless of the original byte sequence.

Example: inserting the message "hi!" produces nodes for '!', 'h', 'i'. After balancing, the inorder traversal returns "!hi".

4. RESEARCH PAPER AND DATA STRUCTURES USED

1. Deep Learning Hierarchical Representations for Image Steganalysis

- Authors: Jian Ye, Jiangqun Ni, Yang Yi
- Published in: IEEE Transactions on Information Forensics and Security (2017)
- Summary: Introduces a CNN architecture that learns noise residuals using fixed high-pass kernels (SRM filters) before feature extraction. Forms the basis of the SRNet design used in this project.
- Link: <https://ieeexplore.ieee.org/document/7937836>

2. Deep Residual Network for Steganalysis of Digital Images (SRNet)

- Authors: Mehdi Boroumand, Mo Chen, Jessica Fridrich
- Published in: IEEE Transactions on Information Forensics and Security (2019)
- Summary: Proposes SRNet, an end-to-end residual CNN that no longer relies on hand-crafted preprocessing for advanced embedding schemes. Inspired the convolutional backbone of our model.
- Link: <https://ieeexplore.ieee.org/document/8470101>

3. Detection of LSB Steganography via Sample Pair Analysis

- Authors: Sorina Dumitrescu, Xiaolin Wu, Zhe Wang
- Published in: IEEE Transactions on Signal Processing (2003)
- Summary: Foundational paper for the Chi-Square sample-pair statistical detector used in our pipeline. Shows that LSB embedding forces $(2k, 2k+1)$ pairs toward equality, making the anomaly detectable with $O(N)$ computation.
- Link: <https://ieeexplore.ieee.org/document/1206551>

4. An Algorithm for the Organization of Information (AVL Trees)

- Authors: G. M. Adelson-Velsky, E. M. Landis
- Published in: Soviet Mathematics Doklady, 1962
- Summary: Original paper introducing the AVL self-balancing binary search tree. Establishes the height-balance invariant and the four rotation cases (LL, LR, RL, RR) used in our payload index.

Data Structures Used

1. AVL Tree (Primary)

A self-balancing Binary Search Tree where, for every node, the heights of the left and right subtrees differ by at most 1. Used to index recovered payload characters.

- Guaranteed $O(\log n)$ insertion, search, and deletion.
- Robust to adversarial input ordering (already-sorted payloads do not degrade to a linked list).
- Inorder traversal yields a deterministic, sorted character stream for forensic fingerprinting.

2. AVLNode

- key: a single character recovered from the LSB stream
- left, right: pointers to child AVL nodes
- height: cached subtree height used for $O(1)$ balance computation

3. PyTorch Tensors and Convolutional Layers

- 4-D tensors of shape (batch, channels, H, W) carry image data through the CNN.
- Fixed Conv2d layer with non-trainable weights implements the SRM high-pass filter bank.
- Trainable ConvBlocks (Conv $\rightarrow$ BatchNorm $\rightarrow$ ReLU) extract hierarchical noise features.

4. NumPy Histogram Arrays

- Two parallel arrays (even_counts, odd_counts) bucket pixel intensities by $v // 2$.
- Vectorised computation of the normalised Chi-Square statistic over qualifying buckets.

Why these data structures?

- AVL Tree for Worst-Case Guarantees: Unlike a plain BST, AVL maintains $O(\log n)$ even on sorted payloads — a realistic adversarial case in steganography (e.g., padded zeros, alphabetical text).
- Tensors for GPU Acceleration: Allow batched CNN inference on CPU/GPU/MPS with no code change.
- Histogram Arrays for $O(N)$ Statistics: Sample-Pair Analysis runs in a single linear pass over the pixels.

5. IMPLEMENTATION

5.1 Code

Repository structure: app.py (Streamlit UI + AVL + extractors), model/srnet.py (CNN + SRM), inference.py (CLI), train.py (training loop).

A. SRM Preprocessing + CNN (model/srnet.py):

```
class SRMFilterLayer(nn.Module):
    """Fixed high-pass filters that suppress image content
    and amplify steganographic noise residuals."""

    def __init__(self):
        super().__init__()
        # 1st-order, 2nd-order, and SQUARE 3rd-order SRM kernels
        filter1 = np.array([[0,0,0,0,0],[0,0,0,0,0],
                           [0,1,-2,1,0],[0,0,0,0,0],
                           [0,0,0,0,0]], dtype=np.float32)
        filter2 = np.array([[0,0,0,0,0],[0,-1,2,-1,0],
                           [0,2,-4,2,0],[0,-1,2,-1,0],
                           [0,0,0,0,0]], dtype=np.float32)
        filter3 = np.array([[-1,2,-2,2,-1],[2,-6,8,-6,2],
                           [-2,8,-12,8,-2],[2,-6,8,-6,2],
                           [-1,2,-2,2,-1]], dtype=np.float32)

        filters = np.stack([filter1, filter2, filter3])
        filters = np.repeat(filters[:, np.newaxis, :, :], 3, axis=1)
        for i in range(filters.shape[0]):
            s = np.abs(filters[i]).sum()
            if s > 0:
                filters[i] /= s

        self.srm_conv = nn.Conv2d(3, 3, kernel_size=5,
                                   padding=2, bias=False)
        self.srm_conv.weight = nn.Parameter(
            torch.from_numpy(filters), requires_grad=False)

    def forward(self, x):
        return self.srm_conv(x)

class StegAnalysisCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.srm = SRMFilterLayer()
        self.features = nn.Sequential(
            ConvBlock(3, 32), ConvBlock(32, 32), nn.AvgPool2d(2),
            ConvBlock(32, 64), ConvBlock(64, 64), nn.AvgPool2d(2),
            ConvBlock(64, 128), ConvBlock(128, 128), nn.AvgPool2d(2),
            ConvBlock(128, 256), ConvBlock(256, 256), nn.AvgPool2d(2),
            ConvBlock(256, 512), ConvBlock(512, 512),
            nn.AdaptiveAvgPool2d(1),
        )
```

```

self.classifier = nn.Sequential(
    nn.Flatten(),
    nn.Dropout(0.5), nn.Linear(512, 128),
    nn.ReLU(inplace=True),
    nn.Dropout(0.3), nn.Linear(128, 1),
)

def forward(self, x):
    x = self.srm(x)
    x = self.features(x)
    return self.classifier(x)

```

B. Multi-Method LSB Extractors (app.py):

```

def _bits_to_text(bits):
    chars = []
    for i in range(0, len(bits) - 7, 8):
        byte_val = 0
        for j in range(8):
            byte_val = (byte_val << 1) | bits[i + j]
        if byte_val == 0:                # null terminator
            break
        if 32 <= byte_val <= 126:
            chars.append(chr(byte_val))
        elif byte_val in (10, 13):
            chars.append(chr(byte_val))
        else:
            break
    text = "".join(chars).strip()
    return text if len(text) >= 2 else None

def extract_raw_lsb_rgb(image):
    """Read LSB of every R,G,B channel in raster order."""
    pixels = list(image.getdata())
    bits = []
    for pixel in pixels:
        for ch in range(3):
            bits.append(pixel[ch] & 1)
        if len(bits) >= 80_000:
            break
    return _bits_to_text(bits)

def extract_raw_lsb_red(image):
    pixels = list(image.getdata())
    bits = [pixel[0] & 1 for pixel in pixels[:80_000]]
    return _bits_to_text(bits)

def extract_raw_lsb_sequential(image):
    """Plane by plane: full Red, then Green, then Blue."""
    pixels = list(image.getdata())
    bits = []

```

```

for ch in range(3):
    for pixel in pixels:
        bits.append(pixel[ch] & 1)
    if len(bits) >= 80_000:
        break
return _bits_to_text(bits)

def extract_all_methods(image):
    for fn in (extract_stegano_lsb,
               extract_raw_lsb_rgb,
               extract_raw_lsb_red,
               extract_raw_lsb_sequential):
        try:
            msg = fn(image)
            if msg:
                return msg
        except Exception:
            continue
    return None

```

C. Chi-Square Sample-Pair Analysis (app.py):

```

def chi_square_lsb_analysis(image):
    arr = np.array(image, dtype=np.int32).flatten()
    if arr.size == 0:
        return 0.0, False

    max_bucket = int(arr.max() // 2) + 1
    even_counts = np.zeros(max_bucket, dtype=np.float64)
    odd_counts = np.zeros(max_bucket, dtype=np.float64)
    for v in arr:
        bucket = v // 2
        if v % 2 == 0:
            even_counts[bucket] += 1
        else:
            odd_counts[bucket] += 1

    total_pairs = even_counts + odd_counts
    mask = total_pairs >= 4
    if mask.sum() == 0:
        return 0.0, False

    expected = total_pairs[mask] / 2.0
    chi_sq = ((even_counts[mask] - expected) ** 2 / expected).sum()
    chi_sq += ((odd_counts[mask] - expected) ** 2 / expected).sum()
    df = mask.sum()
    normalised = chi_sq / df if df else 0.0
    return float(normalised), bool(normalised < 1.5)

```

D. Decision Fusion + AVL Indexing (app.py):

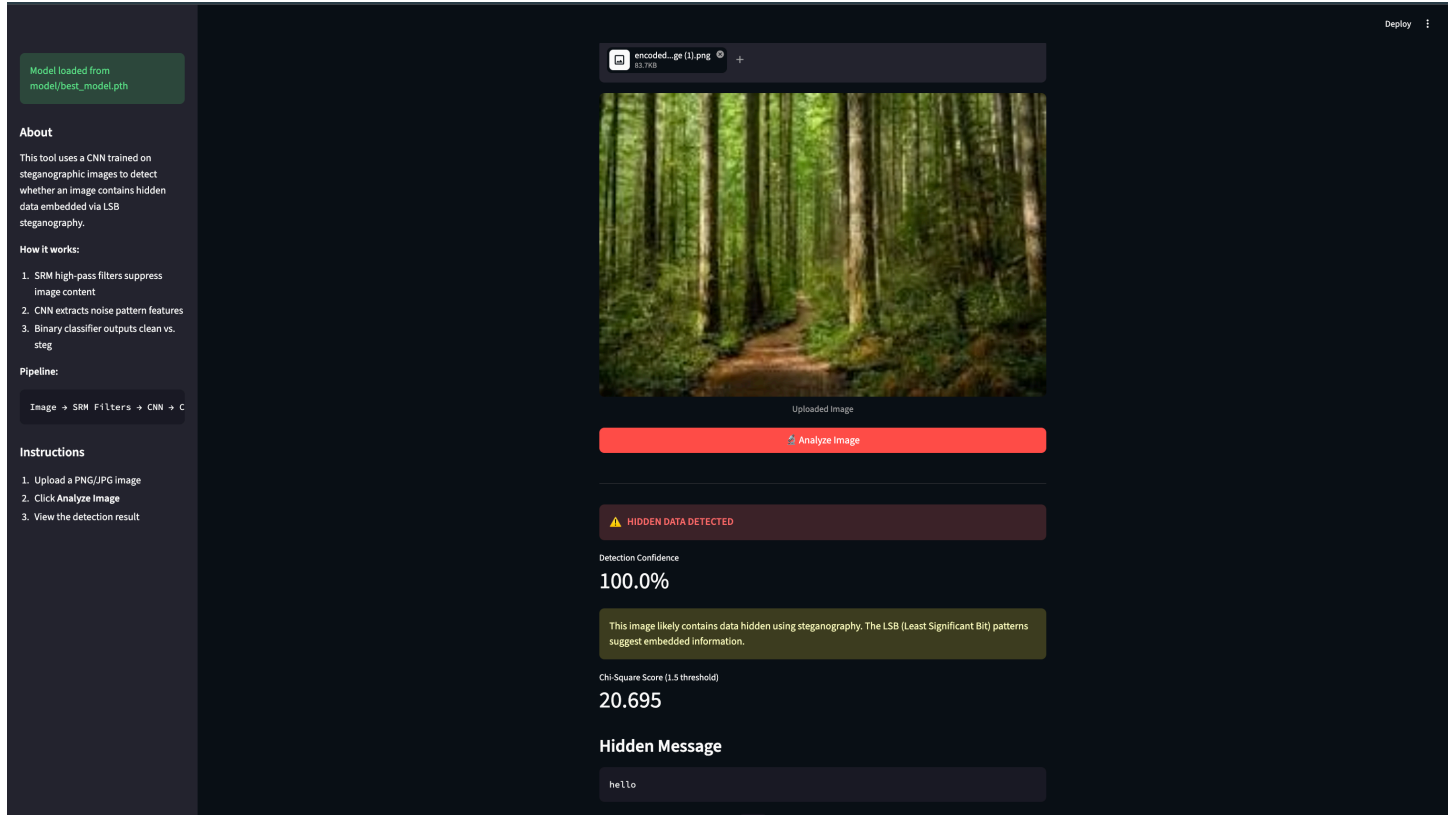
```
is_steg, confidence = predict(model, image_tensor, device)
hidden_message = extract_all_methods(image)
chi_score, is_suspicious = chi_square_lsb_analysis(image)

avl_inorder = ""
if hidden_message:
    is_steg, confidence = True, 1.0          # Definite Threat
    payload_tree = AVLTree()
    for char in hidden_message:
        payload_tree.insert(char)
    avl_inorder = payload_tree.get_inorder()
elif is_suspicious:
    is_steg      = True                      # Likely Threat
    confidence    = max(confidence, 0.90)
```


5.2 Output

The Streamlit web app (streamlit run app.py) presents a single-page interface. After uploading an image and clicking Analyze Image, the dashboard displays the verdict, confidence, Chi-Square score, recovered hidden message, and the AVL inorder traversal of that message.

Sample CLI session (inference.py):

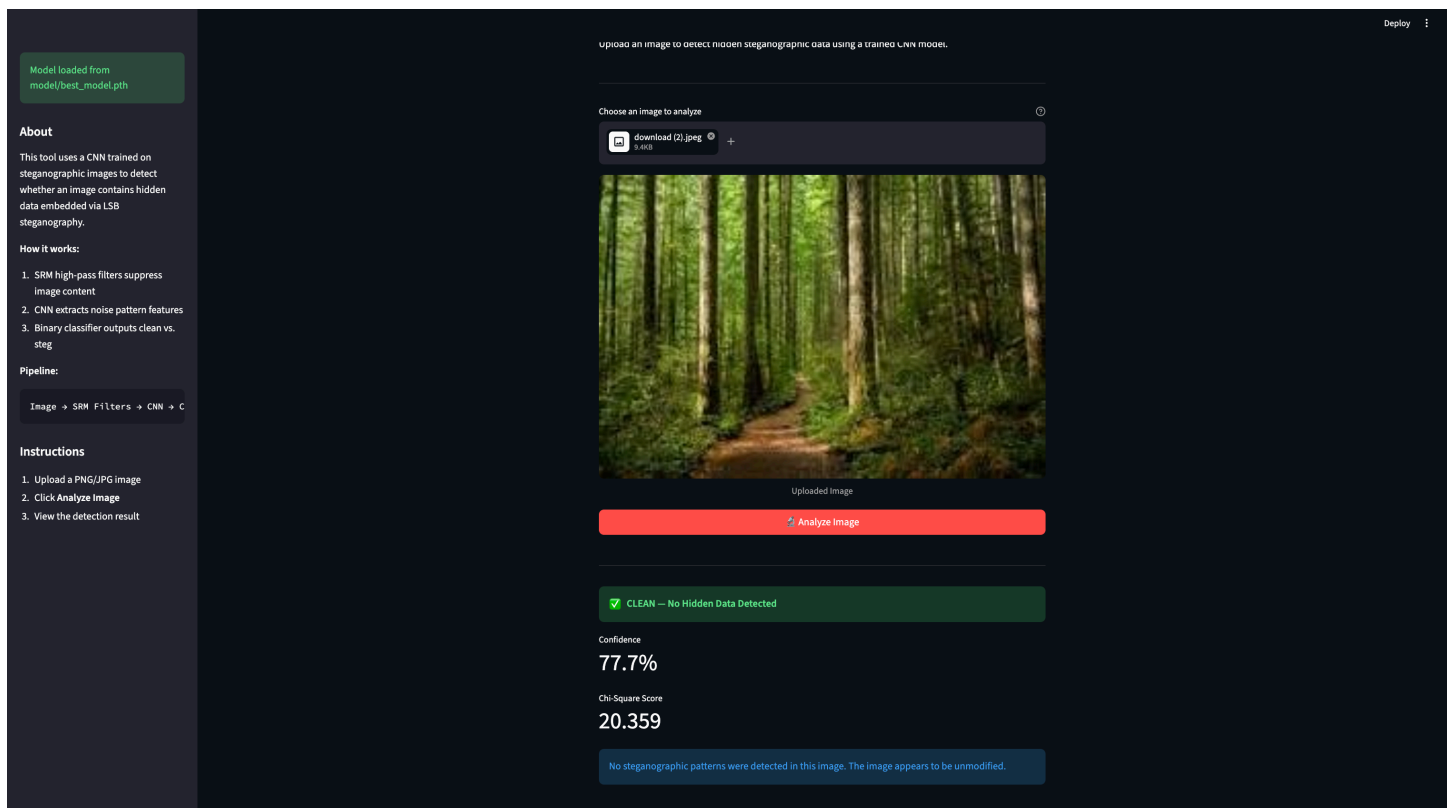


⚠ HIDDEN DATA DETECTED (Confidence: 100.0%)

📄 Hidden message: hello

Chi-Square Score: 20.695 (threshold = 1.5)

Sample clean-image session:



✅ CLEAN (Confidence: 77.7%)

Chi-Square Score: 20.359 (threshold = 1.5)

6. CONCLUSION

The Multi-Layered Image Steganalysis Detector is an efficient solution designed to expose hidden data in digital images by combining a deep CNN with statistical and bit-level forensic checks. Using an SRNet-inspired model with fixed SRM high-pass preprocessing, four independent LSB extraction strategies, a Chi-Square Sample-Pair statistical test, and an AVL Tree payload index, the system delivers fast and high-recall detection of steganographic content from large image datasets.

This system is built with performance and reliability in mind. The SRM preprocessing layer amplifies the minute noise residuals introduced by LSB embedding, while the convolutional backbone learns hierarchical features that are robust to image content. Unlike pure CNN detectors, the multi-method extractor and Chi-Square fallback dramatically reduce false negatives. The AVL Tree guarantees $O(\log n)$ insertion of recovered payload characters even in adversarial cases (already-sorted payloads), providing a deterministic, sortable view of the hidden message for downstream forensic analysis.

The design is highly applicable across multiple domains, including cyber-security operations, digital forensics, malware analysis, content moderation, and intellectual-property protection. These applications require fast and reliable verdicts on suspect media. The decision-fusion engine, inspired by layered defence-in-depth principles, makes the system more resilient than any individual detector for real-time and large-scale environments. Future work will extend the pipeline to PVD spatial steganography, JPEG DCT coefficient analysis, and live AVL Tree visualisation in the frontend.

7. References

- [1] G. M. Adelson-Velsky and E. M. Landis, “An algorithm for the organization of information,” Soviet Mathematics Doklady, vol. 3, pp. 1259–1263, 1962.
- [2] M. Boroumand, M. Chen, and J. Fridrich, “Deep residual network for steganalysis of digital images,” IEEE Transactions on Information Forensics and Security, vol. 14, no. 5, pp. 1181–1193, 2019.
- [3] J. Ye, J. Ni, and Y. Yi, “Deep learning hierarchical representations for image steganalysis,” IEEE Transactions on Information Forensics and Security, vol. 12, no. 11, pp. 2545–2557, 2017.
- [4] S. Dumitrescu, X. Wu, and Z. Wang, “Detection of LSB steganography via sample pair analysis,” IEEE Transactions on Signal Processing, vol. 51, no. 7, pp. 1995–2007, 2003.
- [5] J. Fridrich and J. Kodovsky, “Rich models for steganalysis of digital images,” IEEE Transactions on Information Forensics and Security, vol. 7, no. 3, pp. 868–882, 2012.
- [6] G. Xu, H.-Z. Wu, and Y.-Q. Shi, “Structural design of convolutional neural networks for steganalysis,” IEEE Signal Processing Letters, vol. 23, no. 5, pp. 708–712, 2016.
- [7] A. Westfeld and A. Pfitzmann, “Attacks on steganographic systems,” in Information Hiding, Lecture Notes in Computer Science, vol. 1768, Springer, 2000, pp. 61–76.
- [8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022 — Chapter on Balanced Search Trees.
- [9] PyTorch Documentation, “torch.nn — Neural Network Modules,” <https://pytorch.org/docs/stable/nn.html> (accessed 2026).
- [10] Streamlit Documentation, “Build and share data apps,” <https://docs.streamlit.io> (accessed 2026).